Solving Subset Sum Problems using Quantum Inspired Optimization Algorithms with Applications in Auditing and Financial Data Analysis

1st David Biesner
Fraunhofer IAIS and University of Bonn
Sankt Augustin and Bonn, Germany
david.biesner@iais.fraunhofer.de

2nd Thore Gerlach
Fraunhofer IAIS and University of Bonn
Sankt Augustin and Bonn, Germany

4th Christian Bauckhage
Fraunhofer IAIS and University of Bonn
Sankt Augustin and Bonn, Germany

5th Bernd Kliem
PWC GmbH WPG
Frankfurt a. M., Germany

3rd Rafet Sifa
Fraunhofer IAIS
Sankt Augustin, Germany

arXiv:2211.02653v1 [math.OC] 28 Oct 2022

***Abstract*—Many applications in automated auditing and the analysis and consistency check of financial documents can be formulated in part as the subset sum problem: Given a set of numbers and a target sum, find the subset of numbers that sums up to the target. The problem is NP-hard and classical solving algorithms are therefore not practical to use in many real applications.**

We tackle the problem as a QUBO (quadratic unconstrained binary optimization) problem and show how gradient descent on Hopfield Networks reliably finds solutions for both artificial and real data. We outline how this algorithm can be applied by adiabatic quantum computers (quantum annealers) and specialized hardware (field programmable gate arrays) for digital annealing and run experiments on quantum annealing hardware.[1]

***Index Terms*—QUBO, Quantum Computing, Hopfield Networks, Auditing, Subset Sum**

I. Introduction and Problem Statement

The financial auditing process involves the writing and proofreading of financial reports for the audited company. This process is still largely a manual one: auditors must read documents, compare document content with previous reports, check the completeness of the content to financial regulation checklists and check against both compliance and mathematical errors.

One aspect of mathematical correctness is the correctness of numerical tables, for example describing profit and loss for a given year or quarter. All values in these tables must of course correspond to the actual financial situation of the company, which includes the correctness of sums in the tables. For example, for a table depicting the revenue, expenses and income, the values for expenses and income must sum up to the revenue. Audited reports are provided most commonly as .pdf-files, with no machine readable information on the structure of the sums in the tables available. Even though the numbers in the tables and the corresponding calculations are likely to stem from a table processing software (e.g. Excel) and copied directly, mistakes in the process are possible, e.g. when retroactively updating single values in the tables without updating the corresponding calculations. Therefore, auditors often have to carry out a manual check of table correctness for each document.

During the manual auditing process, one would apply knowledge about financial reports to evaluate which values correspond to which sums for each table and recheck the correctness of the calculations. This is of course highly time and labour intensive, and mistakes are easy to make when auditing a large number of tables.

Automating this process however proves difficult. While a machine has no problem evaluating the correctness of calculations for given table entries, evaluating which values must sum up to which other values is a complex task. Human auditors are either able to apply knowledge on financial reports or knowledge on structure of tables in general: While tables are formatted in a way that human readers can easily evaluate how sums are structured, e.g. by sum values being at the bottom of the tables, indicted by text in the row headers, split from other values by bold lines, teaching a machine to understand these intuitive rules is almost impossible. A strict rule-based approach would be highly dependent on the formatting of specific tables and not generalize well. See Figure 1 for examples of easy and hard tables to parse.

A different approach to this problem is therefore ignoring the table structure altogether. Treating single columns or the entire table as a (ordered) set of numbers, one can try to find which values can be represented as a sum of a subset of all other numbers. This problem can of course be solved exactly by deterministic algorithms. However, the size of tables and magnitude of their entries (for financial documents) make many algorithmic approaches impractical.

In this paper, we evaluate how stochastic algorithms based on gradient descent, minimizing the energy of Hopfield net-

[1]To be published in proceedings of IEEE International Conference on Machine Learning Applications IEEE ICMLA 2022.

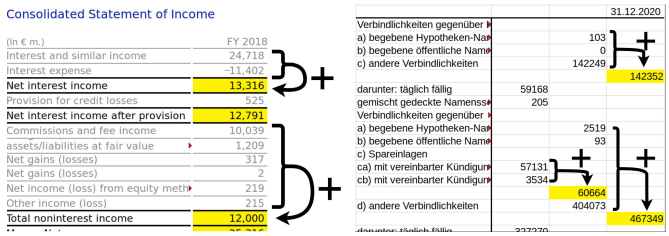

Fig. 1: Left: Example for easy sum structure. Column of a table containing financial performance indicators with annotated sums. Numbers from [1]. Right: Example for more complicated sum structure. Sums extend over multiple columns. Numbers from [2].

works initialized with the problem formulation as parameters, can solve the problem of finding sums in large (both in size and magnitude) tables:

- We restate the problem of finding sums in tables as the subset sum problem and briefly discuss known deterministic solving algorithms.
- We derive the general algorithm for gradient descent on Hopfield networks and how to restate the subset sum problem as a problem solvable by these networks (QUBOs).
- We evaluate our Hopfield algorithm on both artificial and real data and evaluate its performance in the context of financial auditing.
- We describe how quantum annealing hardware is able to solve QUBOs and therefore subset sum problems and run experiments to show that real quantum hardware is capable of solving subset sum problems.

We provide code to reproduce all experiments in this paper at https://github.com/fraunhofer-iais/quantum_subset_sum.

A. Subset Sum Formulation of the problem

We call a set of rules that describe the behaviour of sums in a document, e.g. rows 1 and 2 sum up to row 3, rows 6 to 11 sum up to row 12, as a *sum structure*. See Figure 1 for two examples.

Many tables found in financial reports show the same sum structure in multiple columns, e.g. when comparing financial statements for several quarters and years. Having an efficient algorithm for discovering sums in columns can aid consistency checks by applying the algorithm on one column, extracting a sum structure and checking if the other columns also comply to the found sum structure. If the new column does not comply to the same sum structure, it is an indication for some inconsistency happening in the table. This inconsistency can be displayed to an auditor for further investigation.

If no such repeated structure is present in the table (for example right side of Figure 1), an automatic parsing of sum structures still is valuable. An application can extract all possible sums in the table and present its results to the auditor, who checks whether the parsed sum structure is reasonable, expected sums are missing or the displayed sums do not corespond to actual relations of table items.

Finding sum structures in tables is closely related to a well known problem in algorithmic combinatorics, the subset sum problem. The subset sum problem is defined by a set of numbers $\mathcal{X} = \{x_1, x_2, \ldots, x_n\} \subset \mathbb{Z}$ and a target sum $T \in \mathbb{Z}$. We aim to find a subset $\mathcal{Y}^* \subseteq \mathcal{X}$, such that the sum of the subset is equal to the target sum:

$$\sum_{x \in \mathcal{Y}^*} x = T. \tag{1}$$

In general, due to there being 2^n possible combinations of numbers for the subset, the problem is NP-hard.

In the framework of consistency checks and finding sums in tables, we can consider the entire table as a set of numbers and apply the problem to each entry: taken the entry as a target sum, is it possible to find a subset of all other numbers that sums up to the target? Iterating a solving algorithm over each entry in the table yields a sum structure for the table.

While, there are several known algorithms for solving the subset sum problem, The naive approach consists of cycling through each of the 2^n possible subsets for a total complexity of $\mathcal{O}(2^n n)$. The algorithm can be improved by several heuristics and restrictions on the problem formulation (e.g. only positive integers) but the exponential complexity remains. [3] Dynamic programming algorithms for solving the subset sum problem exactly in pseudo-polynomial time depend on the magnitude of values in $\mathcal{X}$ [4]. Large financial tables defy restrictions on both size and magnitude of values by containing many dozen individual entries and entries of up to billions of euros exact to one cent.

B. Rule-based algorithms for finding sums in financial tables

The problem of finding sum structures in tables does not have to be broken down to the subset sum problem. By ignoring the inherent structure and logic of the table, the complexity of the binary combination problem is increased. Applying rule-based logic and understanding of the general structure of tables can result in efficient algorithms to solve the problem of finding sum structures in tables.

However, rule-based approaches require specialization to each type of table. See right side of Figure 1 for an example of a hard table to parse. Numbers in one column can, but do not have to, correspond to sums in the next column. Membership of a particular number to a column is determined by the accounting related properties of the corresponding value and any approach of parsing the internal table logic into strict rules is very unlikely to generalize to other tables.

C. Related work

Subset sum is a well known special case of the knapsack problem [5]. The run-time complexity of the subset sum problem depends on the parameters n, the number of input values, and M, the precision of the problem, i.e. the number of binary digits necessary to state the problem. All deterministic exact solving algorithms are exponential in either n or M. [6]

There exist dynamical programming approaches to subset sum [7] which offer a polynomial run-time, however they require a space complexity of $\mathcal{O}(nT)$, with T being the target sum. This space complexity is impractical for the proposed application in financial data analysis (see also Section IV-A).

There is much work done on parallelization approaches and GPU-assisted computing to solve an adjusted problem statement in which one tries to maximize $\sum_{x\in\mathcal{Y}} x$ under the restriction $\sum_{x\in\mathcal{Y}} x \leq T$. See [8] for a discussion of various algorithms.

In this work we reformulate the subset sum problem as a QUBO, quadratic unconstrained binary optimization problem. QUBOs are closely related to Ising models and are a natural formulation for many related problems. [9]

We apply optimization over Hopfield networks [10] to optimize an error term derived from the subset sum problem. Hopfield networks are a special type of recurrent neural network which lends itself easily for solving QUBO problems. [11] QUBO problems are also a main application of adiabatic quantum computing, where specialized quantum computers solve QUBOs by a process called quantum annealing. [11]–[13]. In lieu of a quantum computer, specialized non-quantum hardware called field-programmable gate arrays (FPGA) are also able to very quickly and energy-efficiently solve QUBO problems [14].

Lastly, our application lies in the field of automated financial analysis and auditing, a field that has been growing quickly in recent years. Specialized software, for example for text recommendation [15] and automatic anonymization [16] are entering the market and we aim to add further to this toolbox.

II. Subset-Sum as a QUBO

A quadratic unconstrained binary optimization problem (QUBO) is defined by a function $f : \{0,1\}^n \to \mathbb{R}$ which is a quadratic polynomial over its binary input variables,

$$f(\boldsymbol{z}) = \sum_{ij} p_{ij} z_i z_j = \sum_{i\neq j} p_{ij} z_i z_j + \sum_i p_{ii} z_i. \tag{2}$$

The QUBO problem consists of finding the optimal binary vector $\boldsymbol{z}^* \in \{0,1\}^n$ such that

$$\boldsymbol{z}^* = \operatorname*{arg\,min}_{\boldsymbol{z}\in\{0,1\}^n} f(\boldsymbol{z}). \tag{3}$$

The problem can be rewritten in matrix notation as

$$\boldsymbol{z}^* = \operatorname*{arg\,min}_{\boldsymbol{z}\in\{0,1\}^n} \boldsymbol{z}^\intercal \boldsymbol{P}\, \boldsymbol{z} - \boldsymbol{p}^\intercal \boldsymbol{z}$$

with a symmetric and hollow matrix $\boldsymbol{P} \in \mathbb{R}^{n\times n}$ and a vector $\boldsymbol{p} \in \mathbb{R}^n$.

To convert the subset sum problem into a QUBO, we recall the problem statement. Given set $\mathcal{X} = \{x_1, x_2, \ldots, x_n\}$ and target value T, determine a subset $\mathcal{Y}^* \subseteq \mathcal{X}$ such that $\sum_{x\in\mathcal{Y}^*} x - T = 0$.

The subset sum problem can therefore be stated as finding $\mathcal{Y}^*$ such that

$$\mathcal{Y}^* = \operatorname*{arg\,min}_{\mathcal{Y}\subseteq\mathcal{X}} \left(T - \sum_{y\in\mathcal{Y}} y \right)^2. \tag{4}$$

Collecting the numbers contained in set $\mathcal{X}$ in a vector $\boldsymbol{x} = \left[x_1, x_2, \ldots, x_n\right]^\intercal \in \mathbb{R}^n$ and introducing a binary indicator vector $\boldsymbol{z} \in \{0,1\}^n$ with entries

$$z_i = \begin{cases} 1 & \text{if } x_i \in \mathcal{Y} \\ 0 & \text{otherwise} \end{cases}$$

the subset sum problem can alternatively be written as

$$\boldsymbol{z}^* = \operatorname*{argmin}_{\boldsymbol{z}\in\{0,1\}^n} \left(\boldsymbol{x}^\intercal \boldsymbol{z} - T\right)^2.$$

Expanding the equation we write

$$\begin{aligned} \boldsymbol{z}^* &= \operatorname*{arg\,min}_{\boldsymbol{z}\in\{0,1\}^n} \boldsymbol{z}^\intercal \boldsymbol{x}\, \boldsymbol{x}^\intercal \boldsymbol{z} - 2\,T\, \boldsymbol{x}^\intercal \boldsymbol{z} + const \\ &\equiv \operatorname*{arg\,min}_{\boldsymbol{z}\in\{0,1\}^n} \boldsymbol{z}^\intercal \boldsymbol{P}\, \boldsymbol{z} - \boldsymbol{p}^\intercal \boldsymbol{z} \\ &=: \operatorname*{arg\,min}_{\boldsymbol{z}\in\{0,1\}^n} E(\boldsymbol{z}) \end{aligned}$$

where we introduced the shorthands

$$\boldsymbol{P} = \boldsymbol{x}\boldsymbol{x}^\intercal, \quad \boldsymbol{p} = 2\,T\,\boldsymbol{x}. \tag{5}$$

Closely related to QUBOs are Ising Models, where we optimize over $\boldsymbol{s} \in \{-1, 1\}$ instead of $\boldsymbol{z} \in \{0,1\}$:

$$\boldsymbol{s}^* = \operatorname*{arg\,min}_{\boldsymbol{s}\in\{-1,+1\}^n} \boldsymbol{s}^\intercal \boldsymbol{Q}\, \boldsymbol{s} + \boldsymbol{q}^\intercal \boldsymbol{s}$$

Both problem statements are in fact equivalent, with conversion via $\boldsymbol{z} = \frac{1}{2}(\boldsymbol{s} + \boldsymbol{1})$ and $\boldsymbol{s} = 2\boldsymbol{z} - \boldsymbol{1}$.

Converting the QUBO derived from the subset sum problem above, we have

$$\begin{aligned} E(\boldsymbol{z}) &= \boldsymbol{z}^\intercal \boldsymbol{P}\, \boldsymbol{z} - \boldsymbol{p}^\intercal \boldsymbol{z} \\ &= \tfrac{1}{4}\,(\boldsymbol{s}+\boldsymbol{1})^\intercal \boldsymbol{P}\,(\boldsymbol{s}+\boldsymbol{1}) - \tfrac{1}{2}\,\boldsymbol{p}^\intercal(\boldsymbol{s}+\boldsymbol{1}) \\ &= \tfrac{1}{4}\,\boldsymbol{s}^\intercal \boldsymbol{P}\, \boldsymbol{s} + \tfrac{1}{4}\,\boldsymbol{s}^\intercal \boldsymbol{P}\, \boldsymbol{1} + \tfrac{1}{4}\,\boldsymbol{1}^\intercal \boldsymbol{P}\, \boldsymbol{s} - \tfrac{1}{2}\,\boldsymbol{p}^\intercal \boldsymbol{s} + const \\ &= \tfrac{1}{4}\,\boldsymbol{s}^\intercal \boldsymbol{P}\, \boldsymbol{s} + \tfrac{1}{2}\,\boldsymbol{1}^\intercal \boldsymbol{P}\, \boldsymbol{s} - \tfrac{1}{2}\,\boldsymbol{p}^\intercal \boldsymbol{s} + const \\ &\equiv \boldsymbol{s}^\intercal \boldsymbol{Q}\, \boldsymbol{s} + \boldsymbol{q}^\intercal \boldsymbol{s} = E(\boldsymbol{s}), \end{aligned}$$

with the shorthands

$$\boldsymbol{Q} = \tfrac{1}{4}\boldsymbol{P}, \quad \boldsymbol{q} = \tfrac{1}{2}\left(\boldsymbol{P}\,\boldsymbol{1} - \boldsymbol{p}\right), \tag{6}$$

which holds since matrix $\boldsymbol{P} = \boldsymbol{x}\boldsymbol{x}^\intercal = \left(\boldsymbol{x}\boldsymbol{x}^\intercal\right)^\intercal = \boldsymbol{P}^\intercal$ is symmetric and we have $\boldsymbol{s}^\intercal \boldsymbol{P}\, \boldsymbol{1} = \boldsymbol{1}^\intercal \boldsymbol{P}\, \boldsymbol{s}$.

All in all, we can thus consider the subset sum problem as a minimzation problem over $\{-1,1\}^n$ by

$$\boldsymbol{s}^* = \operatorname*{arg\,min}_{\boldsymbol{s}\in\{-1,+1\}^n} \boldsymbol{s}^\intercal \boldsymbol{Q}\, \boldsymbol{s} + \boldsymbol{q}^\intercal \boldsymbol{s} \tag{7}$$

with Q and q defined in (5) and P and p defined in (6).

III. QUBO-Solving with Hopfield Networks

A Hopfield Network is a recurrent neural net of n interconnected neurons. The state of the network is described by a bipolar vector $\boldsymbol{s} \in \{-1, 1\}^n$. Each neuron is connected to every other neuron, with connection weights given by a matrix $\boldsymbol{W} \in \mathbb{R}^{n \times n}$. Moreover, each neuron s_i is a bipolar threshold unit with threshold θ_i, such that

$$s_i = \operatorname{sign}\left(\boldsymbol{w}_t^\intercal \boldsymbol{s} - \theta_i\right). \tag{8}$$

A Hopfield network architecture is therefore fully described by a matrix $\boldsymbol{W}$, a vector $\boldsymbol{\theta}$ and a current state $\boldsymbol{s}$. An update of the network is done via (8), either for all neurons at once or only a subset of neurons.

Defining the energy of the Hopfield network in state $\boldsymbol{s}$ by

$$E(\boldsymbol{s}) := -\frac{1}{2}\boldsymbol{s}^\intercal \boldsymbol{W}\boldsymbol{s} + \boldsymbol{\theta}^\intercal \boldsymbol{s}, \tag{9}$$

we find that, if the weight matrix $\boldsymbol{W}$ is symmetric and is hollow (i.e. has diagonal of all zeros), then the Hopfield energy can never increase when updating one neuron by (8). Since

$$\nabla E(\boldsymbol{s}) = -\boldsymbol{W}\boldsymbol{s} + \boldsymbol{\theta} \tag{10}$$

the updates in (8) amount to $s_i = \operatorname{sign}(-\nabla E(\boldsymbol{s}))$ and each update performs gradient descent on $E(\boldsymbol{s})$. As there are only 2^n possible states the network can be in, successive updates of single neurons will reach a local or global minimum after finitely many updates. See [11] for details of the derivation.

This behaviour can be leveraged to solve problems stated as QUBOs. Encoding the problem in weight and bias parameters $\boldsymbol{W}$ and $\boldsymbol{\theta}$, such that minimum energy states

$$\boldsymbol{s}^* = \underset{\boldsymbol{s} \in \{-1,+1\}^n}{\arg\min} -\frac{1}{2}\boldsymbol{s}^\intercal \boldsymbol{W}\boldsymbol{s} + \boldsymbol{\theta}^\intercal \boldsymbol{s} \tag{11}$$

solve the underlying QUBO problem, the network may find solutions to the QUBO by the described gradient descent updates of single neurons.

To apply Hopfield networks to the subset sum problem, we recall the problem statement as a minimization problem over $\boldsymbol{s} \in \{-1, 1\}^n$ in (7). Defining

$$\boldsymbol{W} = -2\boldsymbol{Q} = -\frac{1}{2}\boldsymbol{P}, \quad \boldsymbol{\theta} = \boldsymbol{q} = \frac{1}{2}(\boldsymbol{P}\boldsymbol{1} - \boldsymbol{p}) \tag{12}$$

we find suitable weights and biases such that the state of a Hopfield network optimized to a global minimum encodes a solution to the subset sum problem.

Note that a Hopfield network with a random initialization does not necessarily converge to a global optimum and local optima are not solutions to the subset sum problem. However, initializing the network multiple times with random states and running until convergence increases the chances of finding a global optimum.

The solving algorithm is easily extensible for efficient utilization of GPUs. See Algorithm 1 for the GPU accelerated full solving algorithm.

Algorithm 1 GPU-accelerated algorithm for solving the subset sum problem with Hopfield networks.

Given a vector of numbers $\boldsymbol{x} \in \mathbb{Z}^n$ and target sum $T \in \mathbb{Z}$,
construct Hopfield network weights $\boldsymbol{W}$ and biases $\boldsymbol{\theta}$ by
$\boldsymbol{W} \leftarrow -\frac{1}{2}\boldsymbol{x}\boldsymbol{x}^\intercal, \quad \boldsymbol{\theta} \leftarrow \frac{1}{2}\left(\boldsymbol{x}\boldsymbol{x}^\intercal \boldsymbol{1} - 2T\boldsymbol{x}\right)$.
Define batch size m dependent on GPU memory.
Define maximum number of steps t_{max}, $t \leftarrow 0$.
while No solution found and $t < t_{\text{max}}$ **do**
  Initialize batch of network states,
  sample $\boldsymbol{S} \in \{-1, 1\}^{\{m \times n\}}$
  while Hopfield energy has not converged **do**
    Calculate gradient:
    $\nabla \boldsymbol{E} \leftarrow -\boldsymbol{W}\boldsymbol{S} + \boldsymbol{1}\boldsymbol{\theta}$
    Select indices of maximum change:
    $\boldsymbol{I}_j \leftarrow \arg\min_i \nabla \boldsymbol{E}_{ji}$
    Update $\boldsymbol{S}_j$ at index $\boldsymbol{I}_j$:
    $\boldsymbol{S}_{j\boldsymbol{I}_j} \leftarrow \operatorname{sign}(\boldsymbol{W}_{\boldsymbol{I}_j}^\intercal \boldsymbol{S}_j - \boldsymbol{\theta}_{\boldsymbol{I}_j})$
    Calculate Hopfield energies of each state in the batch:
    $\boldsymbol{E}_j \leftarrow -\boldsymbol{S}\boldsymbol{W}\boldsymbol{S}_{jj} + (\boldsymbol{\theta}^\intercal \boldsymbol{S})_j$
  end while
  for $j \in \{1, \ldots, m\}$ **do**
    if Hopfield energy is minimal: $\boldsymbol{E}_j = 0$ **then**
      End algorithm.
      return $\{\boldsymbol{x}_i | \boldsymbol{S}_{ji} = 1\}$
    end if
  end for
  $t \leftarrow t + 1$
end while
Algorithm has not found a solution in s_{max} steps.
End algorithm.
return None.

name	M	n	X_{min}	X_{max}
assets	15	17	1.7e+07	1.5e+14
consincome	49	31	−2.5e+12	2.5e+12
liabilities	29	20	2.5e+10	1.5e+14
net revenues	97	25	−5.3e+10	2.5e+12

TABLE I: Configurations of parsed financial data. Each group describes one table in the financial report, with M individual subset sum problems (i.e. sums described in the table), a column length of $n + 1$ which corresponds to n values in the subset sum problem, and values between X_{min} and X_{max}.

IV. Experiments on Hopfield Networks

A. Data

We conduct experiments with both artificial data and real data. To create artificial data we uniformly sample n integers between X_{min} and X_{max}, select k of the sampled integers at random and calculate the target sum T as the sum of the selected integers.

We construct artificial data for $n = 16, 32, 64, 128, 256$ ($k = 4, 8, 8, 8, 8$ respectively) and values in $[-X_{\text{max}}, X_{\text{max}}]$ for $X_{\text{max}} = 1\text{e}{+4}, 1\text{e}{+5}, 1\text{e}{+6}$. For each configuration, we sample $M = 5$ different subset sum problems.

We evaluate our algorithm on a set of real data problems. We parse a financial report [1] containing multiple sheets with financial reports for the quarters from Q1 2019 to Q4 2020 for a total of 190 individual subset sum problems. Each column contains numbers describing amounts up to multiple billion euro, exact to one cent, for a total of 14 significant figures. See Table I for details on the dataset. In contrast to the artificial dataset, the real dataset only contains exactly one solution for most subset sum problems.

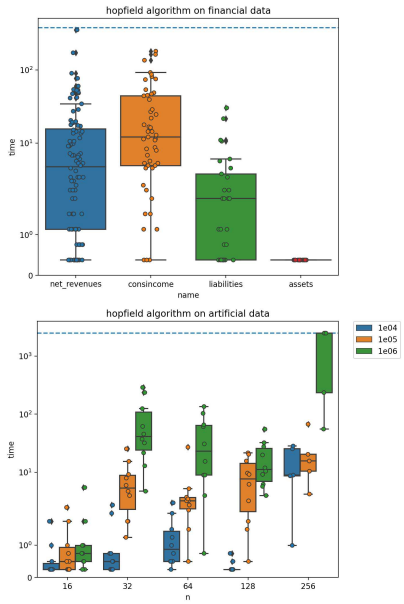

Fig. 2: Benchmarking the Hopfield algorithm on the parsed financial documents (top) and artificial data (bottom). The dotted line roughly indicates the cutoff for computation.

n	X_{max}	t_{mean}	n_{mean}
16	1e+4	0.4	2.0e+4
	1e+5	0.7	3.2e+4
	1e+6	1.1	4.9e+4
32	1e+4	0.7	3.3e+4
	1e+5	6.9	3.1e+5
	1e+6	77.2	3.4e+6
64	1e+4	1.0	4.5e+4
	1e+5	4.9	2.1e+5
	1e+6	40.1	1.7e+6
128	1e+4	0.3	1.3e+4
	1e+5	9.0	3.6e+5
	1e+6	18.4	7.3e+5
256	1e+4	14.4	1.3ee+4
	1e+5	23.5	3.6e+5
	1e+6	145.44	7.3e+5

TABLE II: Results for the Hopfield algorithm on artificial data configurations. The algorithm found a solution for every given sample. For each data configuration we describe the mean time to find a solution in seconds and the mean number of individual runs until a solution is found.

name	n	M	M_{found}	n	mean time	mean runs
assets	17	15	15	17	0.4	1.0e+04
consincome	31	49	49	31	30.4	8.0e+05
liabilities	20	29	29	20	4.1	1.1e+05
net revenues	25	97	97	25	17.6	4.6e+05

TABLE III: Results for the Hopfield algorithm on artificial data configurations. For each data configuration we decribe the mean time to find a solution in seconds and the mean number of individual runs until a solution is found. Of M subset sum problems on each table we solved M_{found}.

B. Experiments and Results

We run the Hopfield algorithm on artificial data for up to 1e+8 initializations in batches of 1e+4 on one NVIDIA A100 GPU, for a maximum computation time of around 8 minutes. For almost all configurations the algorithm reliably finds a correct solution for all samples. Only for the configuration of $n = 256$ and $X_{\text{max}} = 1\text{e}{+}6$ not all samples are solved in the specified maximum number of runs (2 of 5 found). See Figure 2 (bottom) for comparison of computation time against n and X_{max}. We see that the number of values n has a smaller impact on the computation time until a solution is found than the magnitude of the numbers X_{max}.

We run the Hopfield algorithm with the same configuration of the financial data. The algorithm finds a correct solution to the problem under the maximum number of iterations in all cases. Note that unlike for most of the artificial data, the combination of n and X_{max} lead to a situation where each problem likely only contains one correct solution, which is found by our algorithm. See Figure 2 (left) for a comparison of computation time for different tables. We see that average computation time clearly increases with the size of the problem, i.e. amount of values in the table.

See Tables II and III for additional statistics on the runs. In total, we conclude that optimization of binary vectors with Hopfield networks is a reliable algorithm for solving subset sum problems and can be applied to real-world examples.

V. QUBO-Solving with Quantum Annealers

The runtime of all algorithms presented in this paper so far still depends on the complexity of the problem in terms of the amount of numbers in the table. This limitation can be partially erased by application of quantum computing hardware.

A quantum annealer is a type of adiabatic quantum computer that applies the physical quatum annealing process to solve problems. A quantum annealer consists of multiple qubits, which like regular bits can be in the binary states 0 and 1, but also in a superposition between the states.

Solving QUBOs comes naturally to a quantum annealing machine [11]. Each qubit is initialized with an magnetic field called a bias, and multiple qubits can be linked together such that they influence each other, which is called coupling. To solve QUBOs, each qubit represents one binary variable z_i, the bias is initialized by vector $\boldsymbol{p}$ in (6) and the coupling weights are given by matrix $\boldsymbol{P}$ in (6). The initialized system of quantum objects describes an energy landscape and the qubits will naturally find a state of low or minimal energy. At the end of the quantum annealing process, each qubit will collapse into a binary state of either 0 or 1. If the system found a global minimum of the energy landscape, the resulting set of collapsed qbits describes the solution to the QUBO. Due to the physical nature of this process, in theory quantum annealers can find the optimal solution to problems of arbitrary size almost instantly.

However, due to the structure of the graph connecting the qubits, a total of $\mathcal{O}(M)$ physical qubits can only describe $\mathcal{O}(\sqrt{M})$ logical qubits. The most prominent manufacturer of quantum annealers is D-Wave Systems, which provides machines with a maximum of 5436 physical qubits. Meaning the largest quantum annealer available today would be able to solve QUBOs of 70 variables. A smaller machine with 2000

Problem	N_o	N_c	found
#1	1	1	yes
#2	5	1	yes
#3	3	2	yes

Problem	N_o	N_c	found
#4	2	2	yes
#5	8	6	yes
#6	5	5	yes

TABLE IV: Results of the experiments run on a DWave Advantage quantum annealing computer. Each problem stems from a parsed table [18] with $N = 17$ entries of smaller magnitude between $X_{\text{min}} = -2.8\text{e}{+04}$ and $X_{\text{max}} = 4.3\text{e}{+06}$. We evaluate the number of global optima of the energy landscape the annealing process finds (N_{optim}), the number of optima that correspond to a correct solution of the subset sum problem (N_{correct}), and whether one of the proposed solutions corresponds to the correct target solution.

qubits could solve problems of size 50 [17].

We test the capability of current quantum annealers on a small test dataset. In a table [18] with 17 entries of values between $X_{\text{min}} = -2.8\text{e}{+04}$ and $X_{\text{max}} = 4.3\text{e}{+06}$ we identify 6 individual subset sum problems. We convert each problem into a QUBO and run the proposed algorithm on a DWave Advantage quantum annealer with 5436 physical qubits. Table IV describes the results of this experiment. We see that the quantum computer finds the correct proposed solution for each problem.

We run an additional experiment with subset sum problems from a table [19] with 47 entries of $X_{\text{min}} = -9.0\text{e}{+02}$ and $X_{\text{max}} = 2.6\text{e}{+03}$. While the quantum architecture is able to correctly process each QUBO and find solutions for each subset sum problem, the larger amount of numbers with small magnitude leads to issues described in section IV-A, where the number of individual numbers leads to multiple solutions of each target sum, most of which do not correspond to any real-world relationships.

For the time being, quantum annealers are very costly to operate and not entirely practical for this application. However, as technology advances, quantum computing becomes more and more affordable and practical which Once quantum annealing is ready for application on the scale of problems described in this paper, one can directly apply the methodology described in this paper to apply quantum hardware in the auditing process.

VI. Conclusion and Outlook

In this work we investigated how the subset sum problem plays a vital part in the automation of the financial auditing process and how the subset sum problem can be restated as a well known problem architecture which can be solved by the application of gradient descent on the energy landscape of Hopfield networks. We found that the proposed algorithm reliably finds correct sum structures for artificial and real data.

We gave an overview of the capabilities of adiabatic quantum computers for the proposed task and its current limitations. We evaluated the capability of quantum annealers for the subset sum problem and found that for problems with small range of values the algorithm reliable finds correct solutions. As quantum hardware becomes more capable, it will make the use of quantum computing for real applications in finance a realistic possibility.

In the near future, the algorithm will be ready to deploy on existing smart auditing software to directly benefit auditors in their daily work.

VII. Acknowledgements

In parts, the authors of this work were supported by the Smart Data Innovation Challenges (SDI-C) Project "Solving Accounting Optimization Problems in the Cloud", which was funded by the Federal Ministry of Education and Research (BMBF) of Germany.

In parts, this research has been funded by the Federal Ministry of Education and Research of Germany and the state of North-Rhine Westphalia as part of the Lamarr-Institute for Machine Learning and Artificial Intelligence, LAMARR22B.

References

[1] Deutsche Bank, "Financial Data Supplement Q4 2020." https://investor-relations.db.com/files/documents/quarterly-results/4Q20_FDS.xlsx, 2021. Accessed: 2021-12-10.

[2] Deutsche Bank, "Jahres- und Konzernabschluss zum Geschäftsjahr vom 01.01.2020 bis zum 31.12.2020." https://www.bundesanzeiger.de, 2021. Accessed: 2021-12-22.

[3] N. Y. Soma and P. Toth, "An exact algorithm for the subset sum problem," *European Journal of Operational Research*, vol. 136, no. 1, pp. 57–66, 2002.

[4] D. Pisinger, "Linear time algorithms for knapsack problems with bounded weights," *Journal of Algorithms*, vol. 33, no. 1, pp. 1–14, 1999.

[5] H. Kellerer, U. Pferschy, and D. Pisinger, *Knapsack Problems*. Springer, 2004.

[6] E. L. Schreiber, R. E. Korf, and M. D. Moffitt, "Optimal Multi-Way Number Partitioning," *J. ACM*, vol. 65, no. 4, 2018.

[7] K. Koiliaris and C. Xu, "A Faster Pseudopolynomial Time Algorithm for Subset Sum," *arXiv:1507.02318 [cs.DS]*, 2016.

[8] V. Curtis and C. Sanches, "A low-space algorithm for the subset-sum problem on GPU," *Computers & Operations Research*, vol. 83, no. C, 2017.

[9] A. Lucas, "Ising formulations of many np problems," *Frontiers in Physics*, vol. 2, 2014.

[10] C. Bauckhage, R. Ramamurthy, and R. Sifa, "Hopfield networks for vector quantization," in *Proc. ICANN*, Springer, 2020.

[11] C. Bauckhage, R. J. Sánchez, and R. Sifa, "Problem Solving with Hopfield Networks and Adiabatic Quantum Computing," in *Proc. IJCNN*, IEE, 2020.

[12] D-Wave, "What is Quantum Annealing?." https://docs.dwavesys.com/docs/latest/c_gs_2.html, 2021. Accessed: 2021-12-21.

[13] P. Hauke, H. G. Katzgraber, *et al.*, "Perspectives of Quantum Annealing: Methods and Implementations," *Reports on Progress in Physics*, vol. 83, no. 5, 2020.

[14] S. Mücke, N. Piatkowski, and K. Morik, "Hardware Acceleration of Machine Learning Beyond Linear Algebra," in *Proc. ECML/PKDD*, Springer, 2019.

[15] R. Sifa, A. Ladi, *et al.*, "Towards Automated Auditing with Machine Learning," in *Proc. DocEng*, ACM, 2019.

[16] D. Biesner, R. Ramamurthy, *et al.*, "Anonymization of german financial documents using neural network-based language models with contextual word representations," *International Journal of Data Science and Analytics*, Oct 2021.

[17] D-Wave, "The D-Wave Advantage System: An Overview." https://www.dwavesys.com/media/s3qbjp3s/14-1049a-a_the_d-wave_advantage_system_an_overview.pdf, 2021. Accessed: 2021-12-21.

[18] adidas Verwaltungsgesellschaft mbH, "Jahresabschluss zum Geschäftsjahr vom 01.01.2020 bis zum 31.12.2020, BILANZ zum 31. Dezember 2020." https://www.bundesanzeiger.de, 2021. Accessed: 2022-05-02.

[19] adidas AG, "Halbjahresfinanzbericht zum 30. Juni 2019, Konzernkapitalflussrechnung (IFRS) der adidas AG." https://www.bundesanzeiger.de, 2020. Accessed: 2022-05-02.